JC2001: Introduction to Software Engineering

Lecture 6: System Modelling

Lecture Schedule: Week 5

Dr Zhenhua Yu zhenhua.yu@abdn.ac.uk

Dr Riazul Islam riazul.islam@abdn.ac.uk

Dr Shahzad Mumtaz shahzad.mumtaz@abdn.ac.ukc

Learning objectives

- Understand how graphical models can be used to represent software systems.
- Understand the fundamental system modeling perspectives of context, interaction, structure, and behaviour.
- Understand the diagram types in the Unified Modeling Language (UML) and how these diagrams can be used in system modeling.

Reference: Ian Sommerville, Software Engineering – Chapter 5, 10th edition, Pearson, 2015.

Outline of lecture

- Introduction to system modeling.
- Introduction to context models.
- Introduction to interaction models.
- Introduction to the model-driven architecture.

Definition of system modeling

- System modeling is the process of developing abstract models of a system. Each model presents a different perspective of that system. It uses graphical notations based on diagram types in UML.
- System models can be developed for existing systems and new systems:
  - Models of existing systems are used to help clarify what the existing system does. These models can be used to focus a stakeholder discussion on the system's strengths and weaknesses.
  - Models of new systems are used to help explain the proposed requirements to other stakeholders. These models can be used to discuss design proposals and document the system for implementation.

Types of system models

- Different models can be developed for representing the system from different views:
  - An external view of the system using a context or environment model.
  - An interaction view of the system using an interaction model.
  - A structural view of the system using a model of the organisation of a system or the structure of the data processed by the system.
  - A behavioural view of the system using a behavioural model to capture how a system responds to events.

Common use of graphical models

There are three ways in which graphical models are commonly used:

- To focus discussion among software engineers involved in developing a new system or upgrading an existing system. Informal notations may be used in the models.
- To document an existing system. These models document some parts of the system and have to be correct.
- To generate a system implementation. These models are detailed system descriptions, and have to be complete and correct. These models are often used as a basis for generating the source code of the system.

Mock quiz

- What advantage do system models provide when you are developing an existing system?

A. Helps clarify what the existing system does.
B. Helps explain the proposed requirements to other stakeholders.
C. Helps identify gaps in non-functional requirements.
D. Helps determine a suitable programming language.

Context models

- At an early stage in the specification of a system, you should decide on the system boundaries, that is, on what is and is not part of the system being developed.
- This involves working with system stakeholders to decide what functionality should be included in the system, and what processing and operations should be carried out in the system's operational environment. For example, you may decide that:
  - Automated support for some business processes should be implemented in the software being developed.
  - Some new functionality should be implemented to address overlaps in fuctionality with existing systems.

Mentcare system: Context model

- The Mentcare system is intended to manage information about patients attending mental health clinics, and the treatments that have been prescribed. In developing the specification for this system, you have to decide whether the system should focus on one of the following:
    - Collecting information about consultations using other systems to collect personal information about patients, or
    - The system should also collect personal patient information.

- The advantage of relying on other systems for patient information is that it avoids duplicating data.
- The disadvantage is using other systems may make it slower to access information.

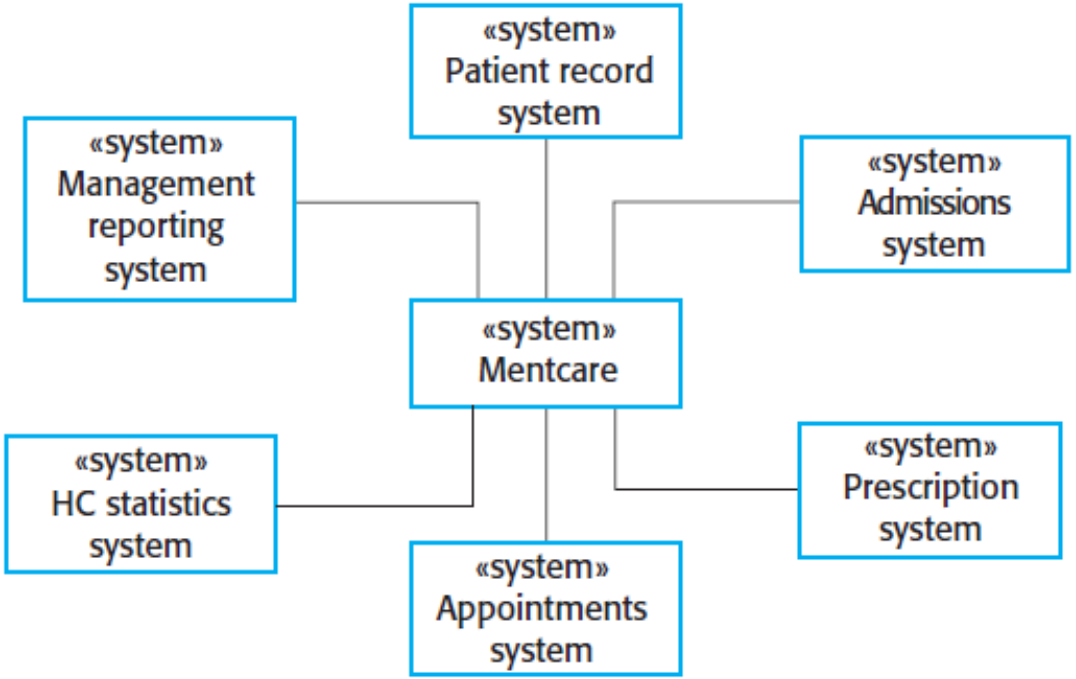
Mentcare system: Context model

- The user base for the Mentcare system is diverse, and users have a wide range of different system requirements.
- The list of users include doctors, nurses, receptionists, medical ethics advisor, etc. To cater to the needs of different users, a configurable system that can be adapted to their needs can be developed.

To define the context and the dependencies that a system has on its environment, a simple architectural model can be used.

Mentcare system: Context model

- An architectural model for the Mentcare system.



Mock quiz

- The Mentcare system pulls patient records from an existing patient information system. While this approach simplifies system development, it also raises some issues. Discuss one such issue.

USS Capita data breach – April 2023

- USS members national insurance number, title, data of birth, name were stolen.
- Breach was suspected to have occurred when data was transferred between different servers.
- Data breach cost Capita upto $25 million.
- UK lawyers consider bringing a joint claim over the data breach.



Capita cyber-attack: USS pension fund members' details may have been stolen

Universities Superannuation Scheme says it can not be certain information about 470,000 members is safe

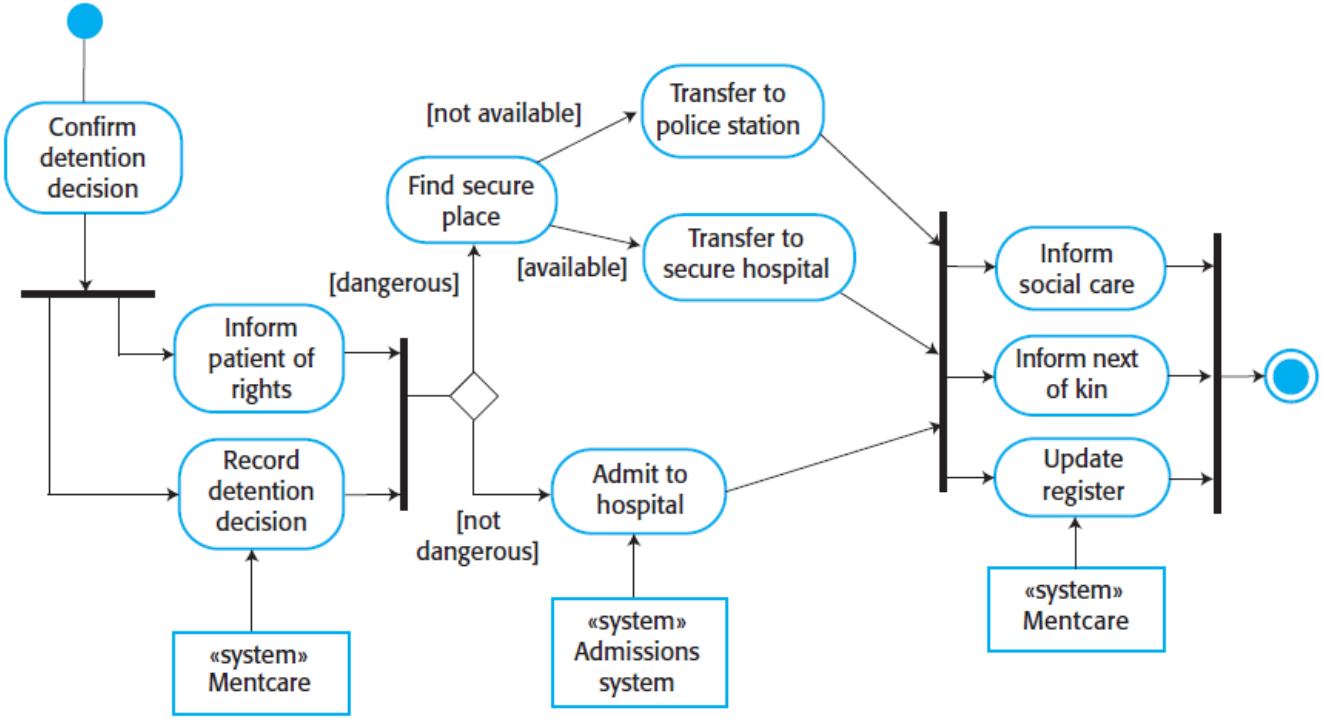
Source: www.theguardian.com

Business process model

- Business process models are used along with simple context models. Business process models describe human and automated processes in which particular software systems are used.
- Context models normally show that the environment includes several other automated systems. However, context models do not show the types of relationships between the systems in the environment, and the system that is being specified.
- External systems might produce data for or consume data from the system. They might share data with the system, or they might be connected directly, through a network or not connected at all. They might be physically co-located or located in separate rooms or buildings. All these relations may affect the requirements and design of the system being defined, and so must be taken into account.

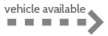
Mentcare system: Process model

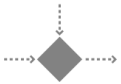
- A business process model of patient detention, showing by an UML *activity diagram*.



1495 UNIVERSITY OF ABERDEEN

Symbolic representation in UML activity diagrams

	Start	Used at the beginning or start of a proess. Usually just one start in an activity diagram.
	End	Used at the end of a process. There could be multiple ends within a process to show the end of each branch of the process.
	Activity	Represents an activity or task. There will be multiple tasks within an activity diagram. Named using the verb-noun naming convention. A task represents one person doing one thing at one point in time.
	Flow arrow	Represents the flow of the process. Links different shapes together flowing in the direction of the arrow.
	Annotated Arrow	Same as a flow arrow but includes annotation to add more information to a diagram,
	Flow arrow	Represents the flow of the process. Links different shapes together flowing in the direction of the arrow.
	Swim Lane	Represents the flow of the process. Links different shapes together flowing in the direction of the arrow.

	Decision	Represents where the flow follows a different path depending on the result of the decision. The process will only follow one of the outgoing paths from a deicion
	Merge	Represents where two or more paths converge back into one path. Only one of the incoming paths needs to hit the merge for the process to continue.
	Fork	Represents where the process splits in to different paths, unlike a decision the process will follow all outgoing paths.
	Join	Represents where two or morepaths converge back into the same path. All of the incoming paths must reach this point before the process continues.
	Time	Used to depict where time impacts the process such as a passage of time or a date in the calendar.
	Outgoing Event	Used where the process creates an event but the outcome of the event is outside the scope of the activity diagram.
	Incoming Event	Used were the process receives an event from another process or source that is outside the scope of the activity diagram.
	Annotation	Used to add text to annotate or elaborate on parts of the diagram.

1495 UNIVERSITY OF ABERDEEN

Interaction models

- All systems involve interaction of some kind. Examples of such interactions include the following:
  - User interaction, which involves user inputs and outputs.
  - Interactions between the software being developed and other systems in its environment.
  - Interactions between the components of a software system.
- Interaction modeling is important for the following reasons:
  - It helps to identify user requirements.
  - It highlights communication problems that may arise in system-to-system interactions.
  - It helps to understand if a proposed system structure is likely to deliver the required system performance and dependability.

Mock quiz

- Which of the following are reasons for modeling interactions in a computer system?

A. It helps in identifying system requirements.
B. It highlights communication problems that may arise in system-to-system interactions.
C. It helps in understanding if the proposed system structure is likely to deliver the required system performance and dependability.
D. All of the above.

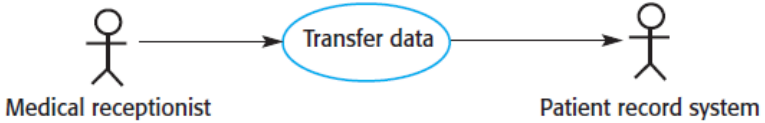
Interaction modeling approaches

- There are two related approaches to interaction modeling:
  - ***Use case modeling***, which is mostly used to model interactions between a system and external agents (human users or other systems).
  - ***Sequence diagrams***, which are used to model interactions between system components, although external agents may also be included.

- Use case models and sequence diagrams present interactions at different levels of detail, and so they may be used together. For example, the details of the interactions involved in a high-level use case may be documented in a sequence diagram.

Mentcare system: Use case model

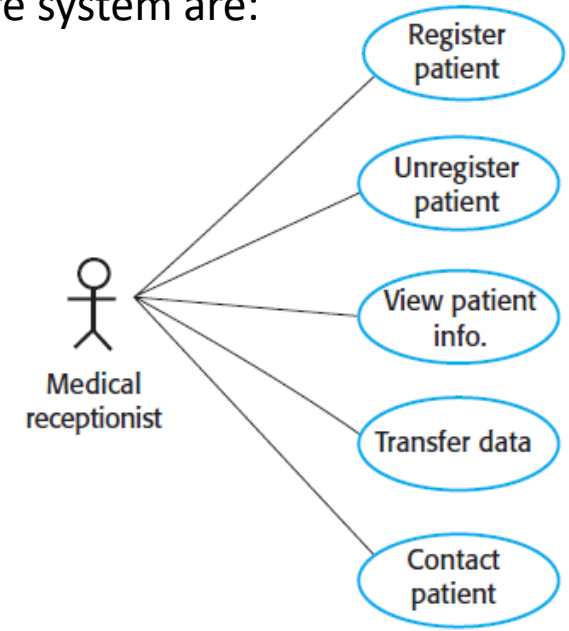
- A high-level use case and description for transfer-data.



Mentcare system: Transfer data	
Actors	Medical receptionist, Patient records system (PRS)
Description	A receptionist may transfer data from the Mentcare system to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary
Stimulus	User command issued by medical receptionist
Response	Confirmation that PRS has been updated
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

Mentcare system: Composite use case diagram

- Composite use case diagrams show a number of different use cases. For example, some use cases in the Mentcare system are:
  - Register patient.
  - Unregister patient.
  - View patient information.
  - Transfer data.
  - Contact patient.



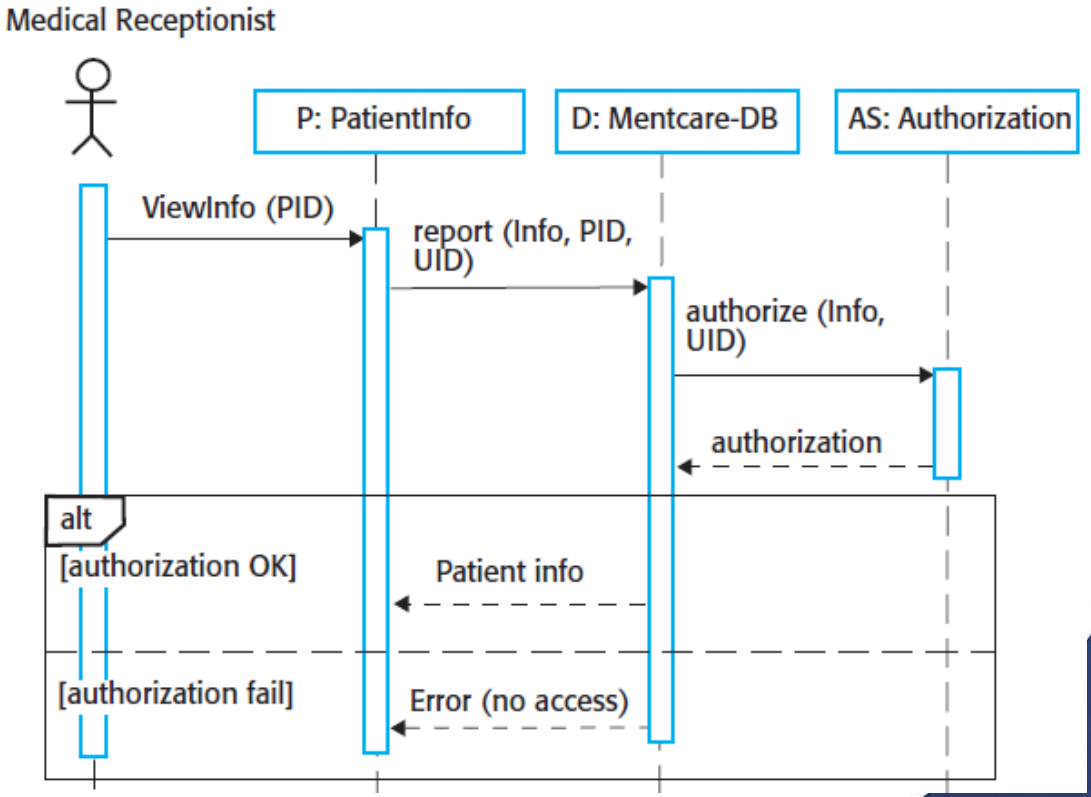
Sequence diagrams

- Sequence diagrams in the UML are primarily used to model the interactions between the actors and the objects in a system, and the interactions between the objects themselves.
- A sequence diagram shows the sequence of interactions that take place during a particular use case. The structure of a sequence diagram is given as follows:
  - The actors and objects involved are listed along the top of the diagram.
  - Annotated arrows indicate interactions between objects. The annotations on the arrows indicate the calls to the objects, their parameters, and the return values.
  - The rectangle on the dotted lines indicates the lifeline of the object concerned.
  - You read the sequence of interactions from top to bottom.

Mentcare: Sequence diagram for View patient info

This sequence diagram can be read as follows:

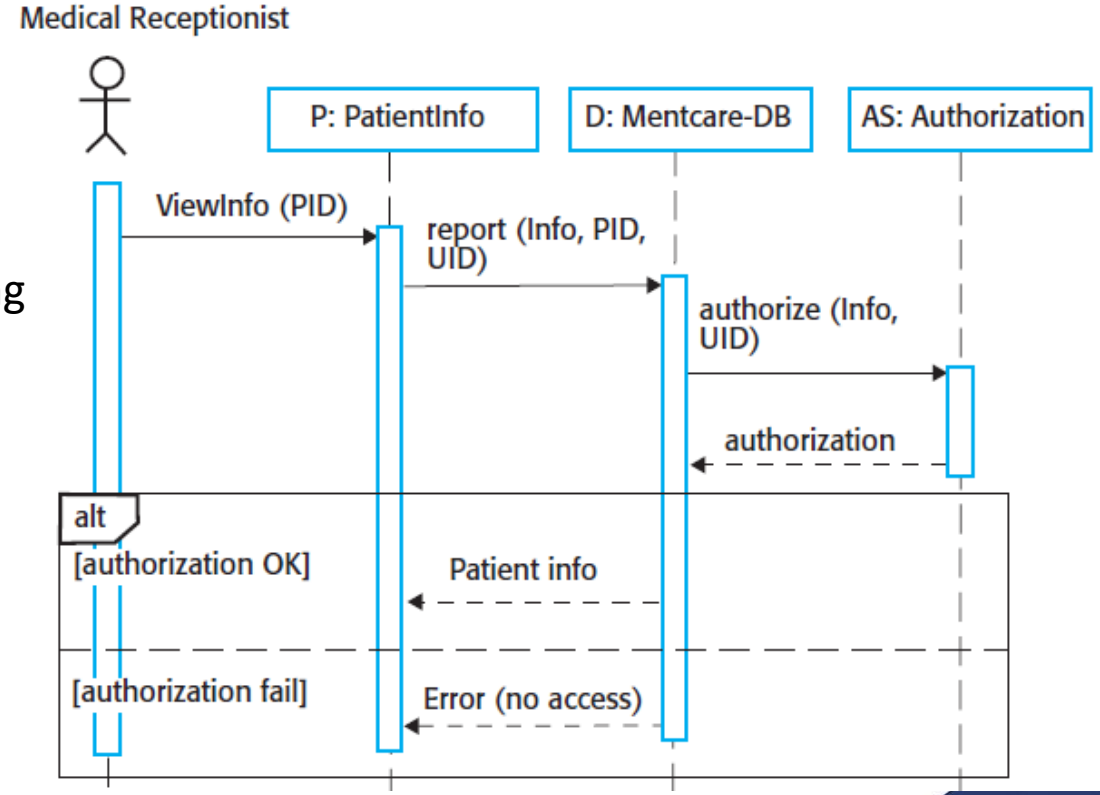
- The medical receptionist triggers the ViewInfo method in an instance of P of the PatientInfo class, supplying the patient's identifier, PID to identify the required information. P is a user interface object, which is displayed as a form showing patient information.



Mentcare: Sequence diagram for View patient info

This sequence diagram can be read as follows (cont'd):

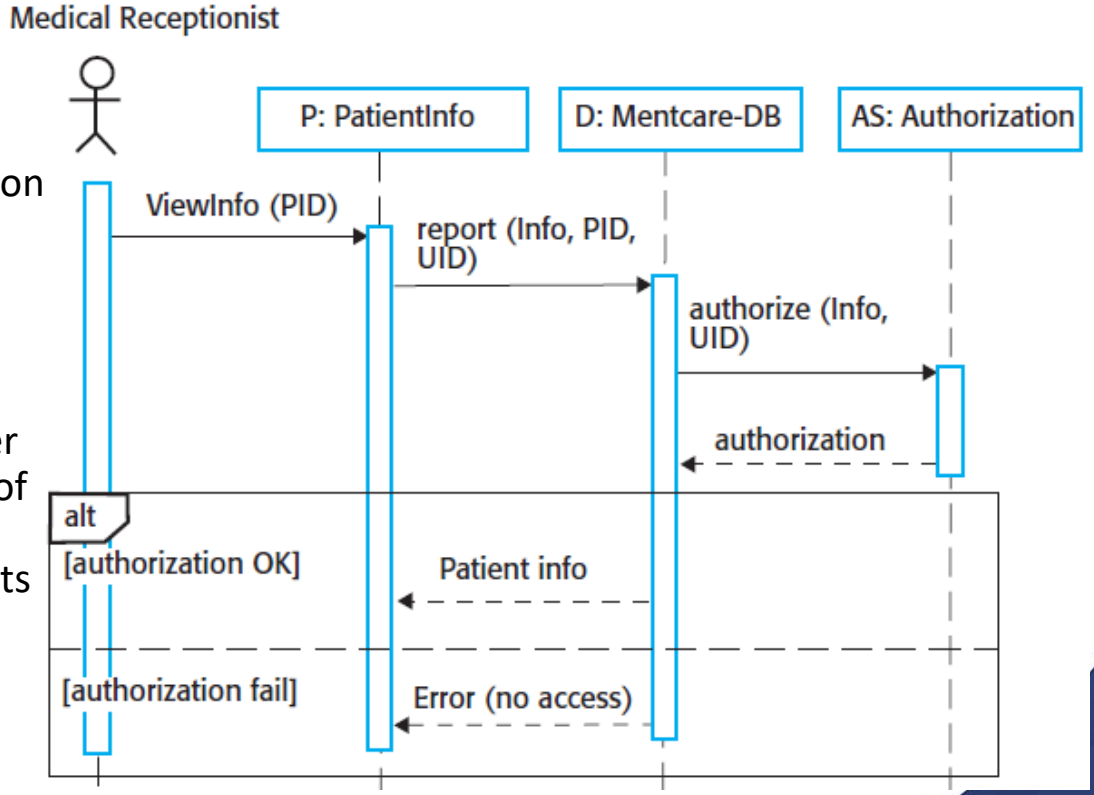
- The instance P calls the database to return the information required, supplying the receptionist's identifier to allow security checking.
- The database checks with an authorisation system that the receptionist is authorised for this action.



Mentcare: Sequence diagram for View patient info

This sequence diagram can be read as follows (cont'd):

- If authorised, the patient information is returned and is displayed on a form on the user's screen.
- If authorisation fails, then an error message is returned. The box denoted ``alt'' in the top-left corner is a choice box indicating that one of the contained interactions will be executed. The condition that selects the choice is shown in square brackets.



Sequence diagrams

- Unless you are using sequence diagrams for code generation or detailed documentation, you don’t have to include every interaction in these diagrams.
- If you develop system models early in the development process to support requirements engineering and high-level design, there will be many interactions that depend on implementation decisions.

Event-driven modeling

- Event-driven modeling shows how a system responds to external and internal events. It is based on the assumption that a system has a finite number of states and that events may cause a transition from one state to another. For example, a system controlling a valve may move from a state ``Valve open'' to a state ``Valve closed'' when an operator command is received. Event-driven modeling is used extensively when designing and documenting real-time systems.

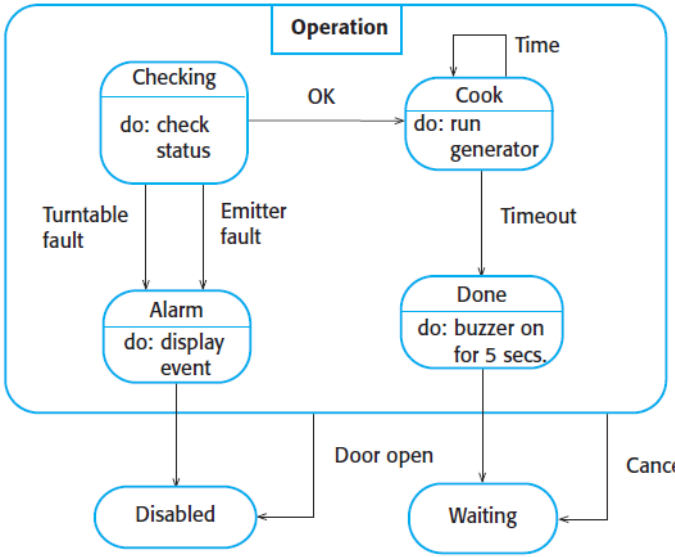
- In UML, event-based modeling is supported using state diagrams, which are based on state charts. State diagrams show system states and events that cause transitions from one state to another. They do not show the flow of data within the system, but may include additional information on computations carried out in each state.

State model

- A simple oven has a switch to select full or half power, a numeric keypad to input the cooking time, a start/stop button, and an alphanumeric display.
- Using the microwave oven:
  - Select the power level.
  - Input the cooking time using a numeric keypad.
  - Press Start and the food is cooked for the given time.

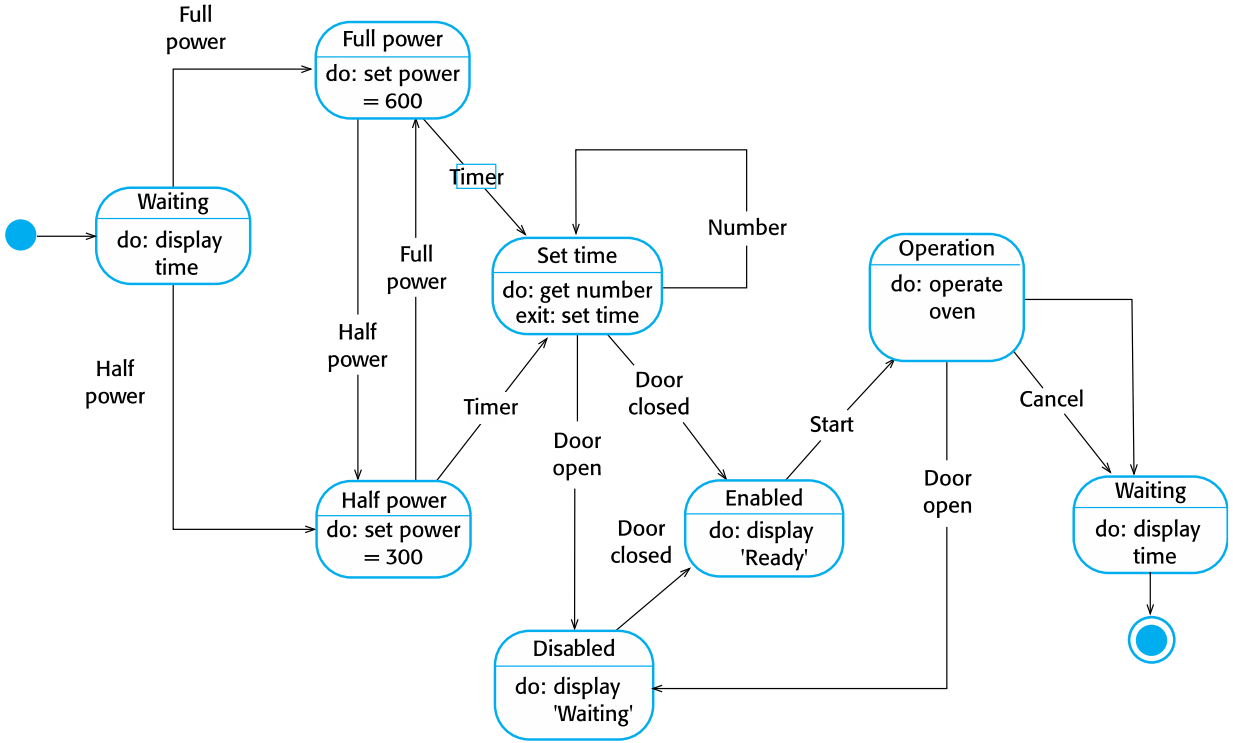


- State-based modeling is susceptible to the problem of rapid increase in the number of possible states. To address this issue, ``super states'' that encapsulate a number of separate states can be used.



- Round rectangles represent system states.
- Labeled arrows represent events.

State model



1495 UNIVERSITY OF ABERDEEN

State description and stimulus

- State models provide an overview of event processing, but a more detailed description of the system states and stimuli is required for extending the state models.

State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows "Half power."
Full power	The oven power is set to 600 watts. The display shows "Full power."
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows "Not ready."
Enabled	Oven operation is enabled. Interior oven light is off. Display shows "Ready to cook."
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for 5 seconds. Oven light is on. Display shows "Cooking complete" while buzzer is sounding.

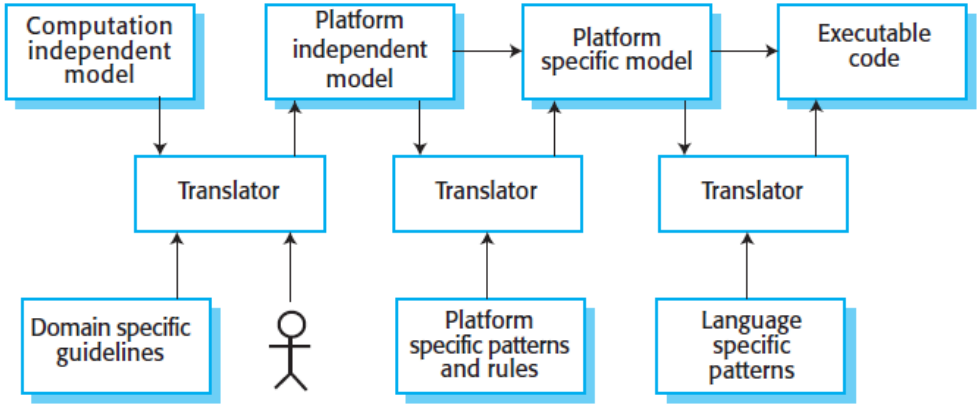
Stimulus	Description
Half power	The user has pressed the half-power button.
Full power	The user has pressed the full-power button.
Timer	The user has pressed one of the timer buttons.
Number	The user has pressed a numeric key.
Door open	The oven door switch is not closed.
Door closed	The oven door switch is closed.
Start	The user has pressed the Start button.
Cancel	The user has pressed the Cancel button.

Model-driven architecture

- Model-driven architecture (MDA) is a model-focused approach to software design and implementation that uses a subset of UML models to describe a system. Models at different levels of abstraction are created.

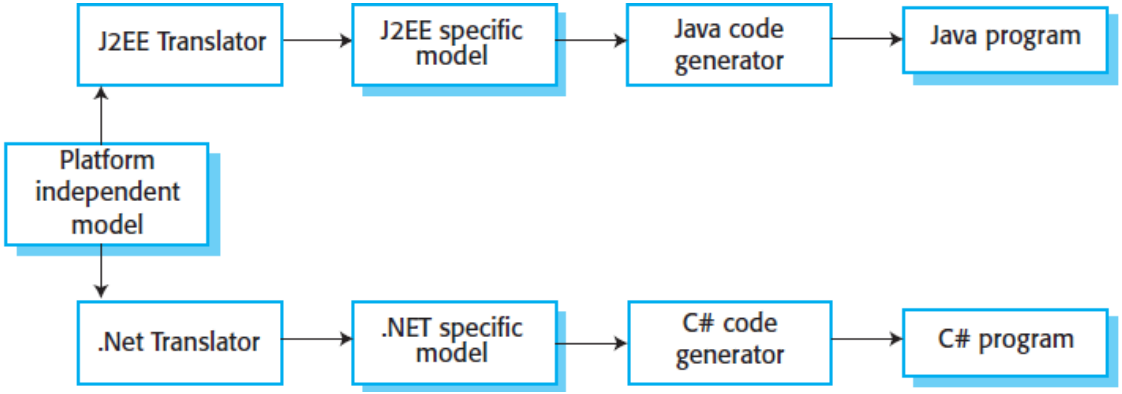
- The MDA method recommends that three types of models should be produced:
  - Computation independent model. These models model the important domain abstractions used in a system, and are sometimes called domain models.
  - Platform-independent model. These models model the operation of the system without reference to its implementation.
  - Platform-specific model. These models are transformations of the platform-independent model with a separate platform-specific model for each application platform.

MDA transformations



- Fundamental to MDA is the notion that *transformations* between models can be defined and applied automatically by software tools.
- A translator is applied to models to generate execution code that will run on the designated software platform. However, it is difficult to automatically transform models into execution code, so the need for human intervention.

MDA transformations



- The translation of platform-independent to platform-specific models is a simpler technical problem. Commercial tools and open-source tools are available that provide translators from platform-independent models to common platforms such as Java, J2EE, .NET, etc. An extensive library of platform-specific rules and patterns can be used to transform these models.

Lessons learned

- We described the system modeling process, described the types of system model, and their uses.
- We gave details of context and interaction models, and presented examples of these models.
- We described the model-driven architecture and its various models.

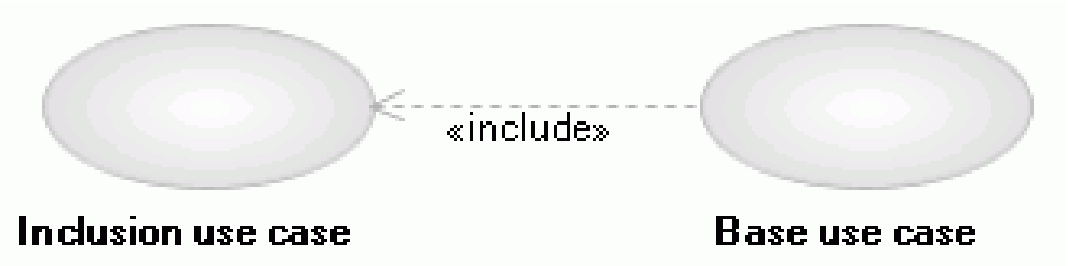
Additional Slides on System Modelling



1495 UNIVERSITY OF ABERDEEN

System Modelling: Include relationships

A relationship in which one use case (the base use case) includes the functionality of another use case (the inclusion use case).



https://www.ibm.com/docs/en/dma?topic=diagrams-include-relationships

System Modelling: Include relationships

The include relationship supports the reuse of functionality in a use-case model.

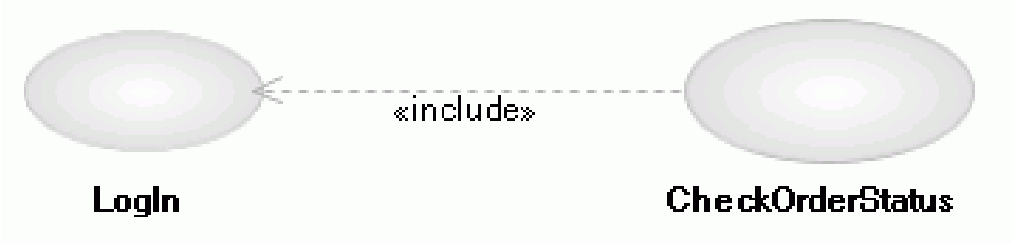
Useful to show the following situations:

- The behavior of the inclusion use case is common to two or more use cases.
- The result of the behavior that the inclusion use case specifies, not the behavior itself, is important to the base use case.

Include relationships usually do not have names.

System Modelling: Include relationships

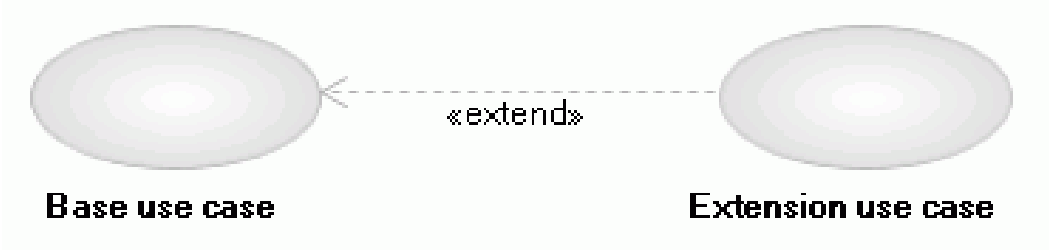
Example: An e-commerce application that provides customers with the option of checking the status of their orders.



a base use case called **CheckOrderStatus** that **has an inclusion use case** called **LogIn**

System Modelling: Extend relationships

A relationship to specify that one use case (extension) extends the behavior of another use case (base).



This type of relationship reveals details about a system or application that are typically hidden in a use case.

System Modelling: Extend relationships

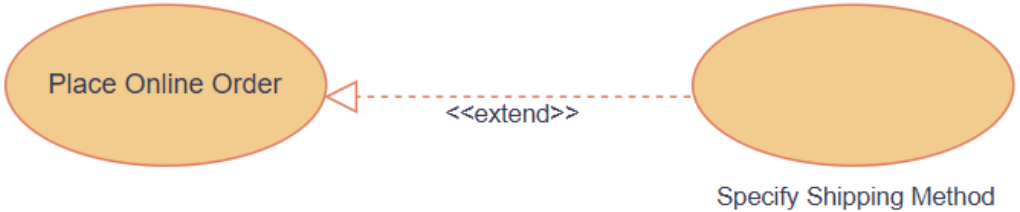
- The extend relationship specifies that the incorporation of the extension use case is dependent on what happens when the base use case executes.
- The extension use case owns the extend relationship.
- You can specify several extend relationships for a single base use case.

Useful to show the following situations:

- A part of a use case that is optional system behavior
- A subflow is executed only under certain conditions
- A set of behavior segments that may be inserted in a base use case

System Modelling: Extend relationships

Example: In an e-commerce system in which you have a base use case called Place Online Order that has an extending use case called Specify Shipping Instructions.



indicate that the behaviors in the Specify Shipping Instructions use case are optional and only occur in certain circumstances.

Design a Use Case Diagram for a University Library System

Description

- Students can search for books, borrow books, and return them.
- Librarians manage book records and check overdue fines.
- The system automatically sends reminders for overdue books.

Task

1. Identify the actors involved.

2. Identify at least five use cases based on the description.

3. Draw a Use Case Diagram showing: Actors, Use cases, System boundary, and Relationships (include / extend if needed).

Design a Use Case Diagram for a University Library System

Description

- Students can search for books, borrow books, and return them.
- Librarians manage book records and check overdue fines.
- The system automatically sends reminders for overdue books.

Task

1. Identify the actors involved.

2. Identify at least five use cases based on the description.

3. Draw a Use Case Diagram showing: Actors, Use cases, System boundary, and Relationships (include / extend if needed).

Design a Use Case Diagram for a University Library System

Description

- Students can search for books, borrow books, and return them.
- Librarians manage book records and check overdue fines.
- The system automatically sends reminders for overdue books.

Task

1. Identify the actors involved.

2. Identify at least five use cases based on the description.

3. Draw a Use Case Diagram showing: Actors, Use cases, System boundary, and Relationships (include / extend if needed).